

Deep Learning Optimisé - Jean Zay

Optimizing large models part 2



INSTITUT DU
DÉVELOPPEMENT ET DES
RESSOURCES EN
INFORMATIQUE
SCIENTIFIQUE

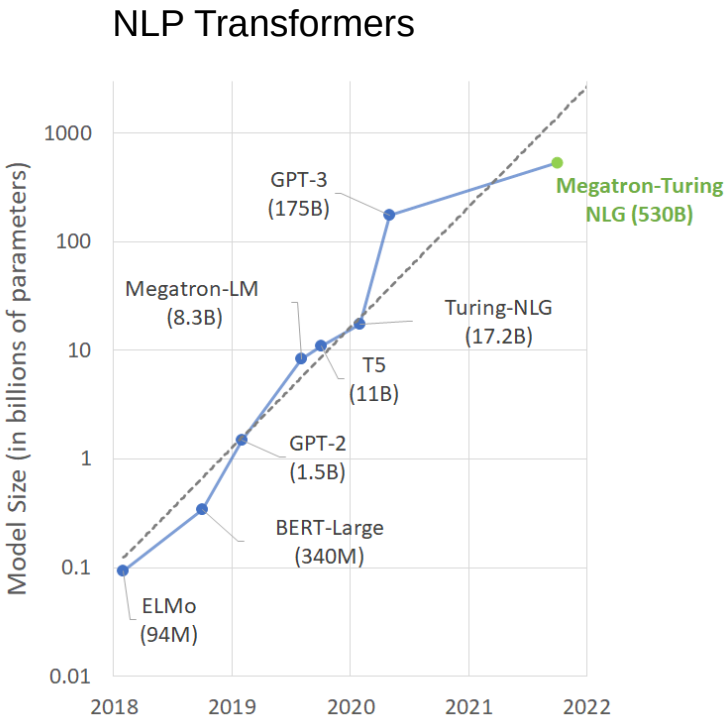
cc BY NC SA

Data Parallelism Optimization for Very Large Models

Challenges of Large Models ◂

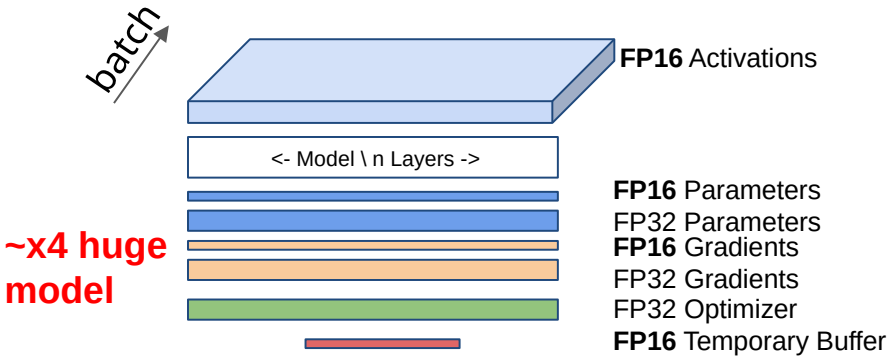
ZeRO & Fully Sharded Data Parallelism ◂

Huge models > 1 billion parameters



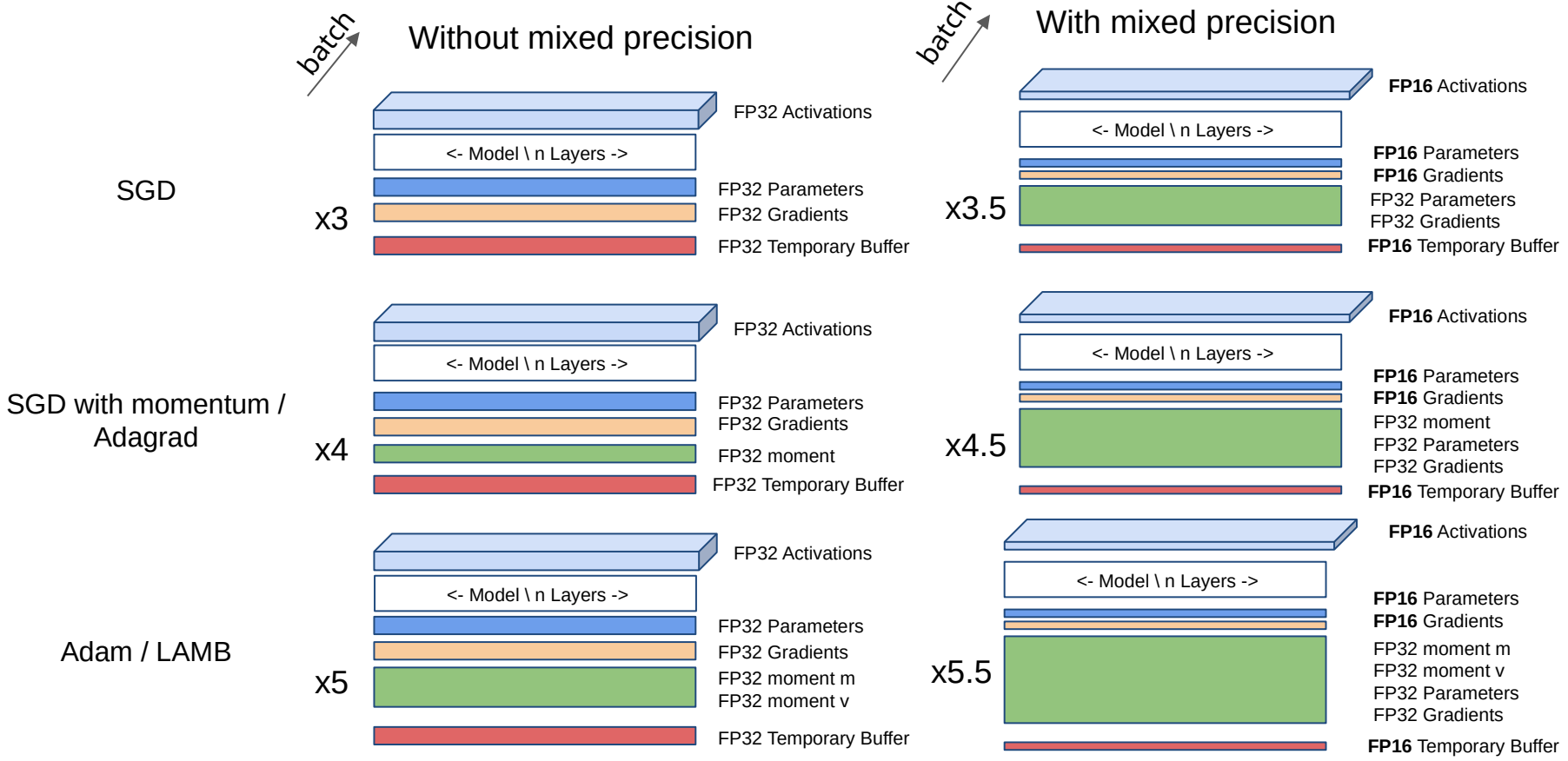


CUDA Out Of Memory

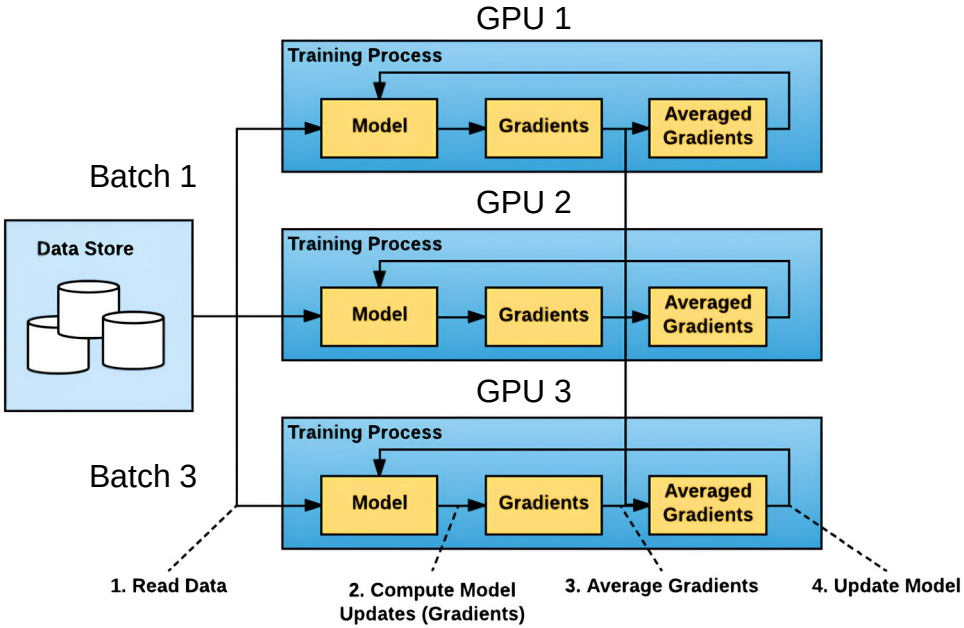


[1]

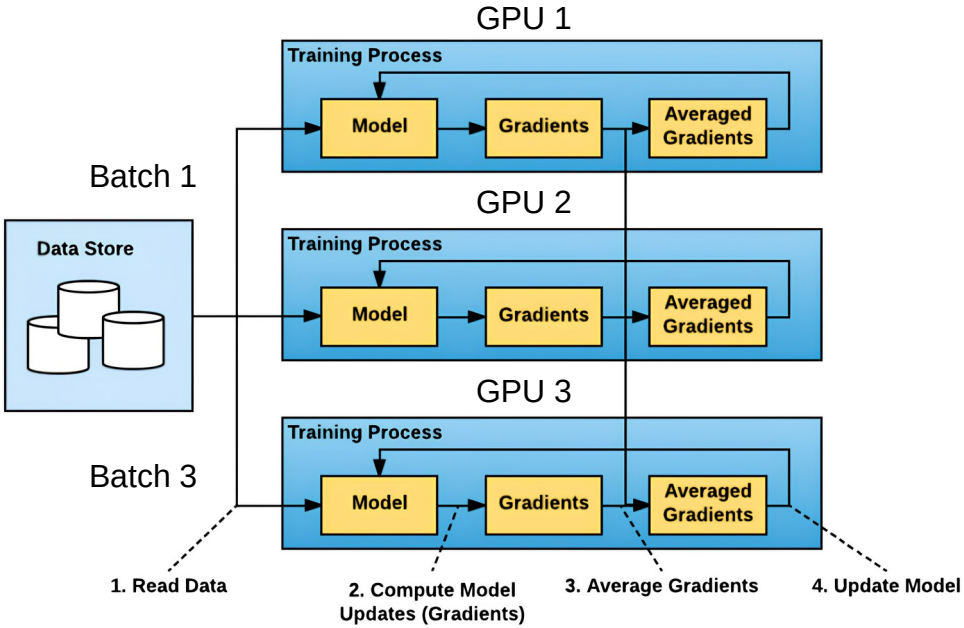
Memory Footprint

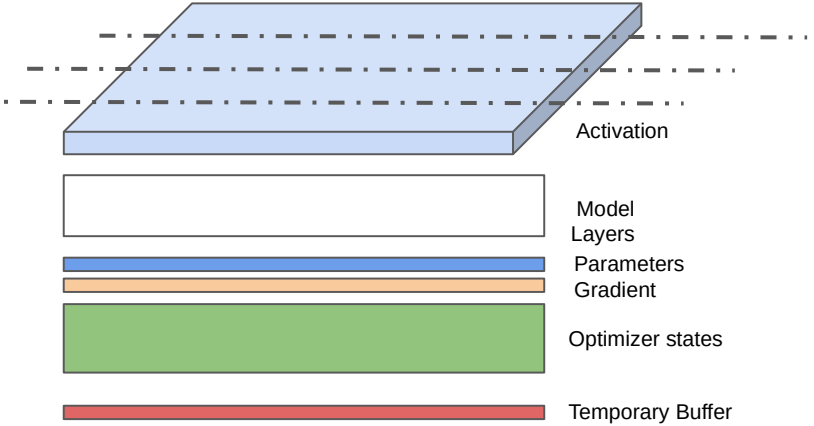


Reminder : Data Parallelism

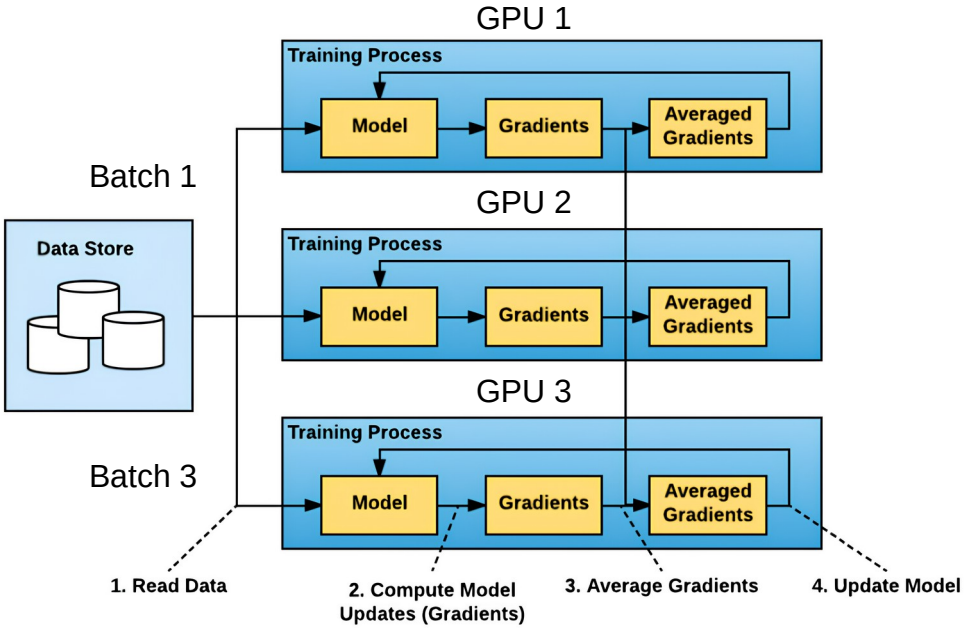


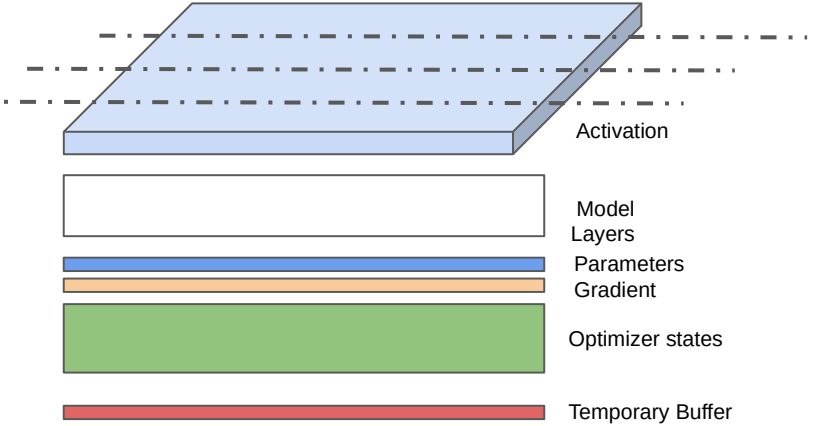
Reminder : Data Parallelism





Reminder : Data Parallelism

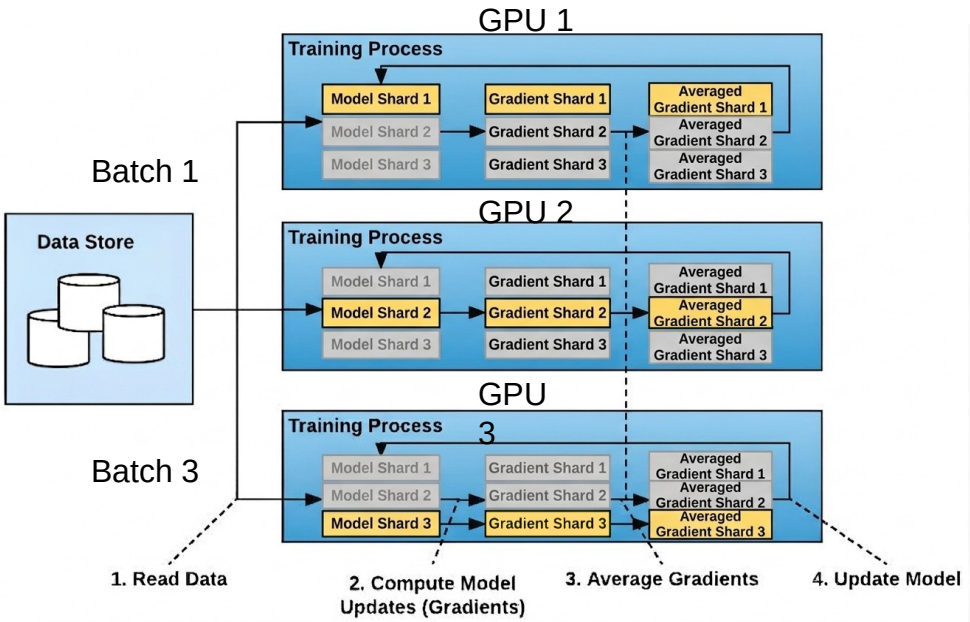




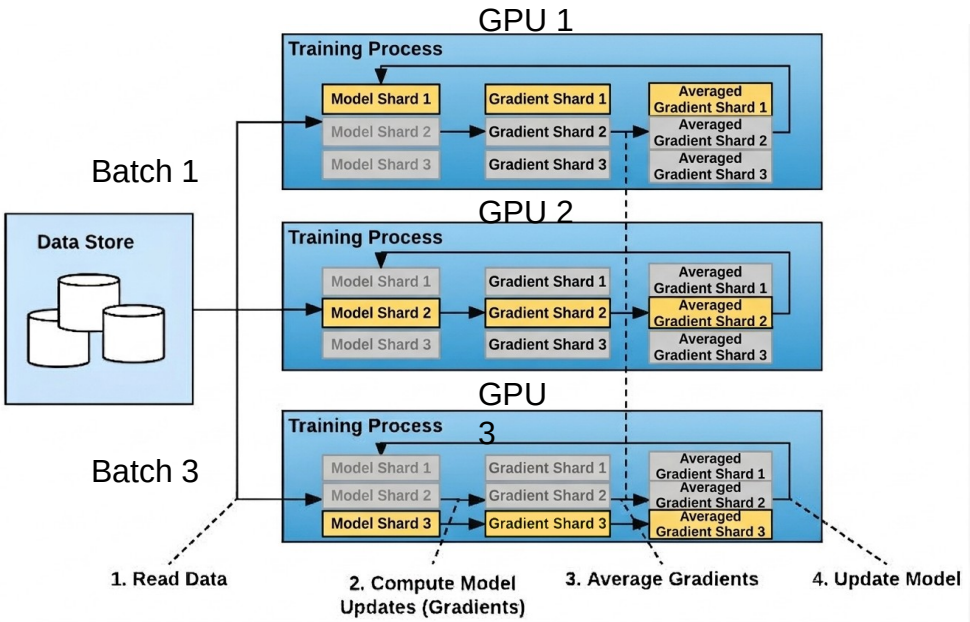
Advantages
-High Throughput
-Only the activation memory footprint is distributed
-Multi-processing

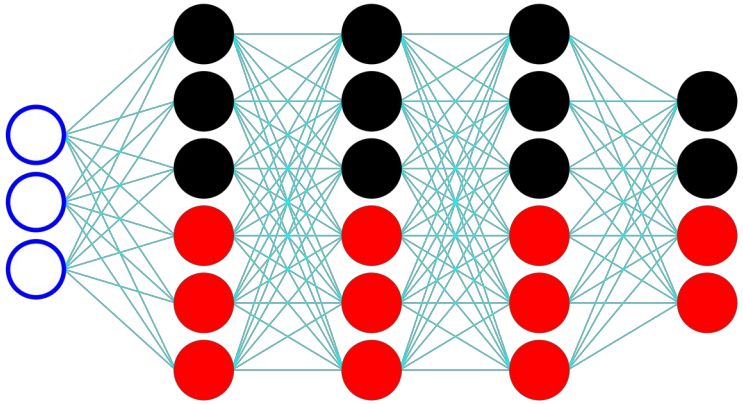
Drawback
-Each GPU must be able to fit the full model in VRAM.

Fully Sharded Data Parallelism

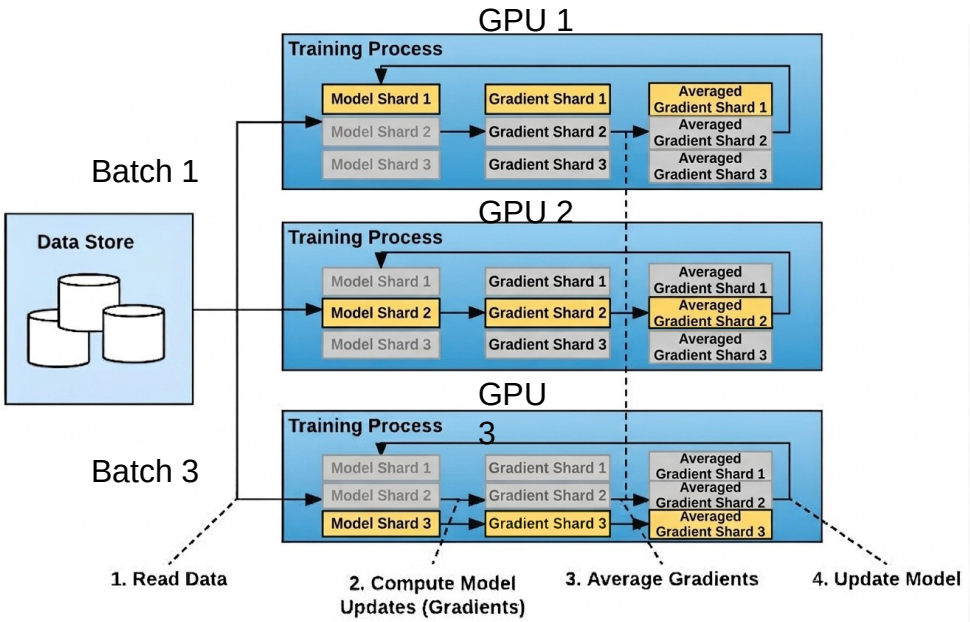


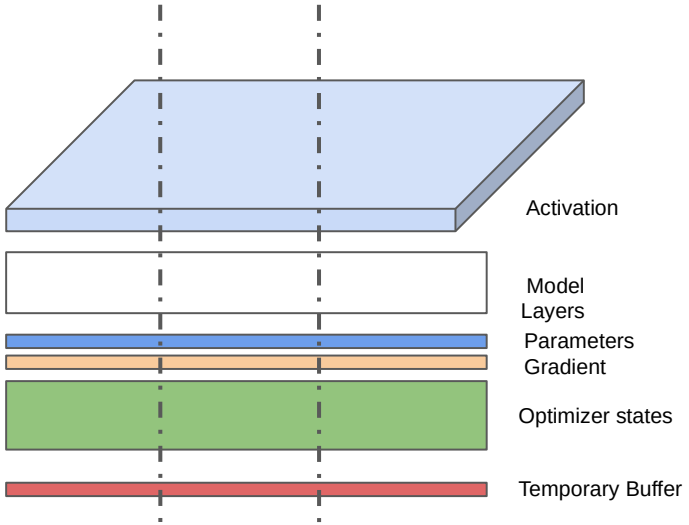
Fully Sharded Data Parallelism



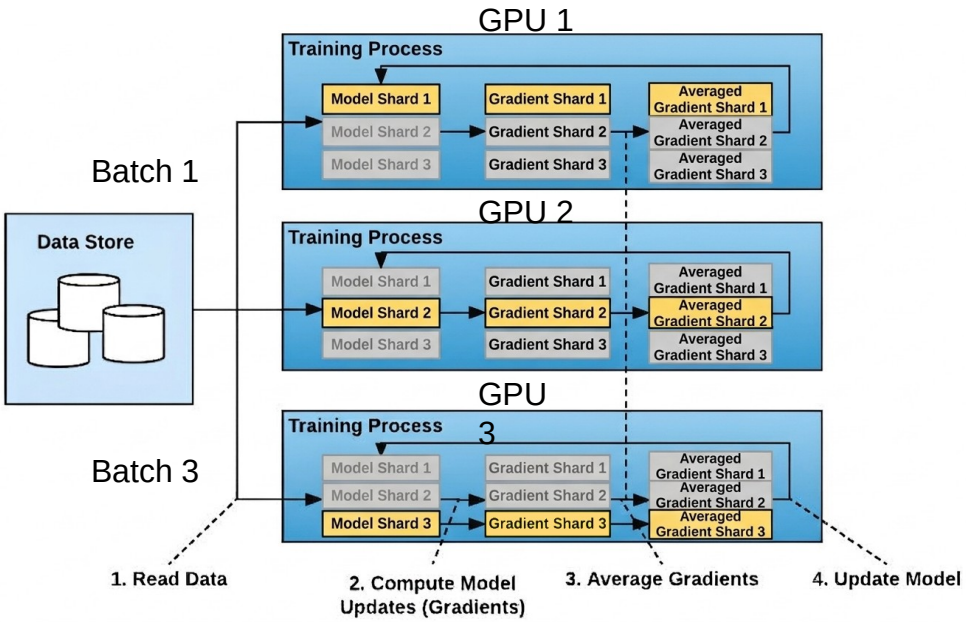


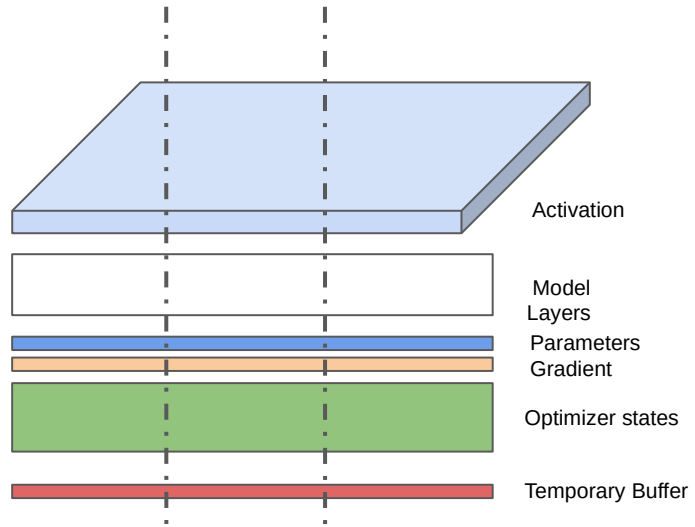
Fully Sharded Data Parallelism





Fully Sharded Data Parallelism





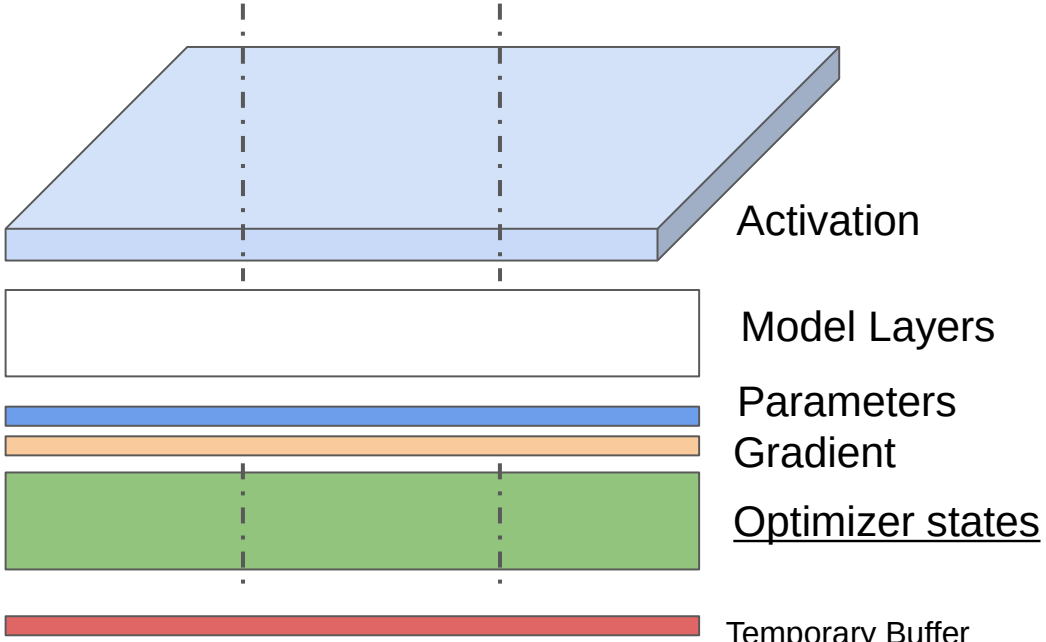
Advantage

-Lets you run models larger than your VRAM

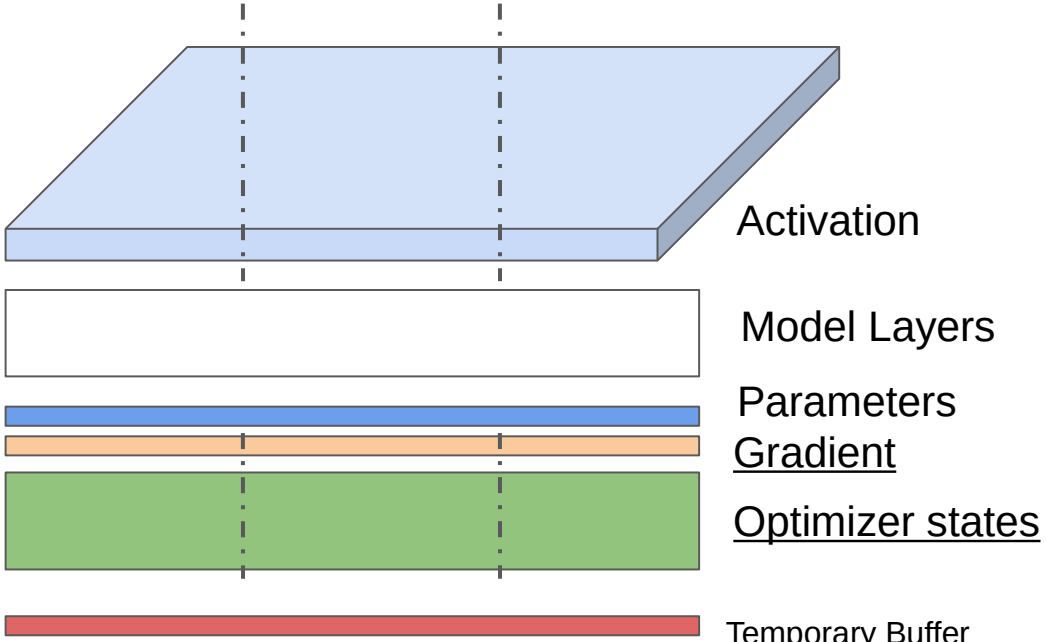
Drawback

-FSDP generates a lot of communication

Alternative to FSDP : ZERO 1



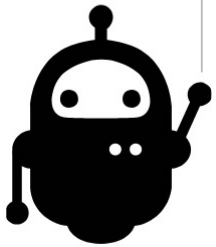
Alternative to FSDP : ZERO 2



TP : Fully Sharded Data Parallelism







Go to the tp_fsdp directory in the Jour4 directory of the course repository and follow the notebook.

- Limitations of Data Parallelism with Llama 3.2 3B
- Implement Fully Sharded Data Parallelism (FSDP)
- Optimize FSDP
- Use torch.compile with an FSDP module

Optimizing the learning of very large models

Pipeline parallelism ◂

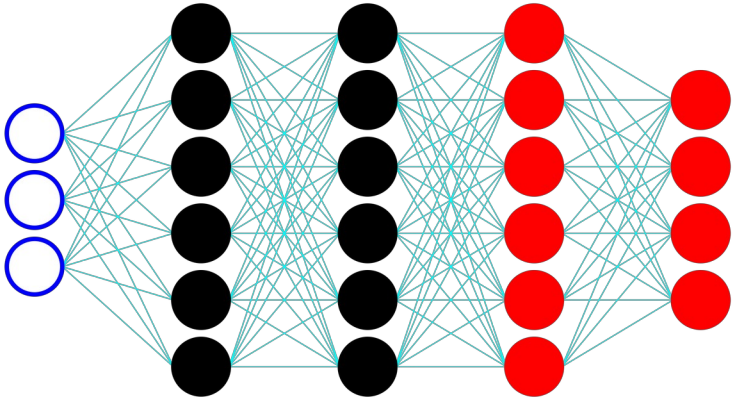
Tensor parallelism ◂

Hybrid parallelism ◂

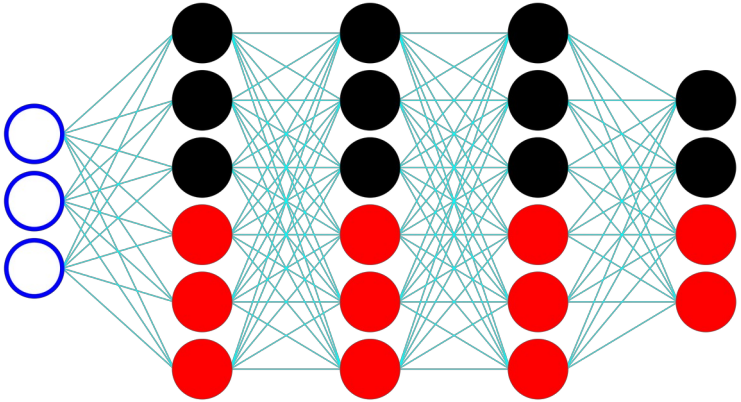
3D parallelism ◂

Two paradigms for model parallelism

Pipeline Parallelism



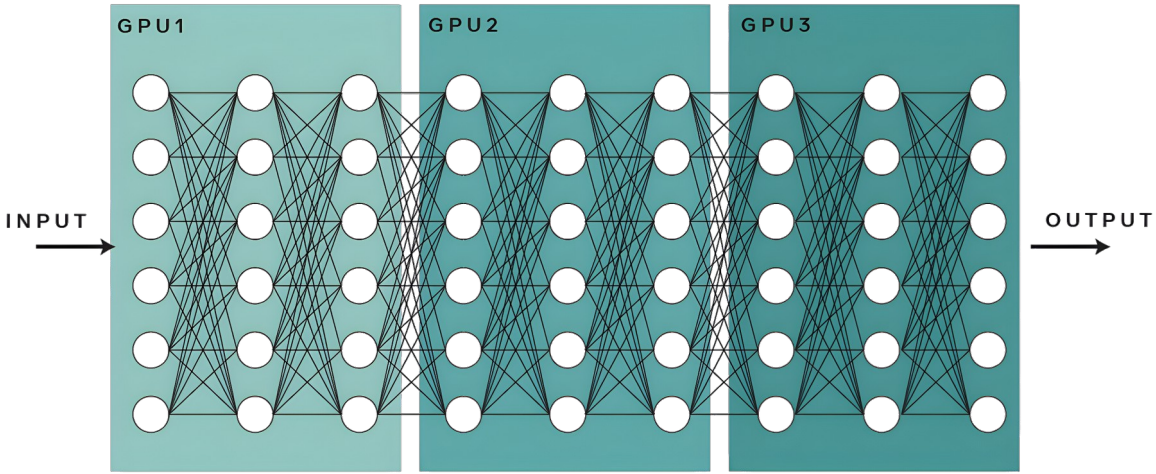
Tensor Parallelism and FSDP



GPU 0

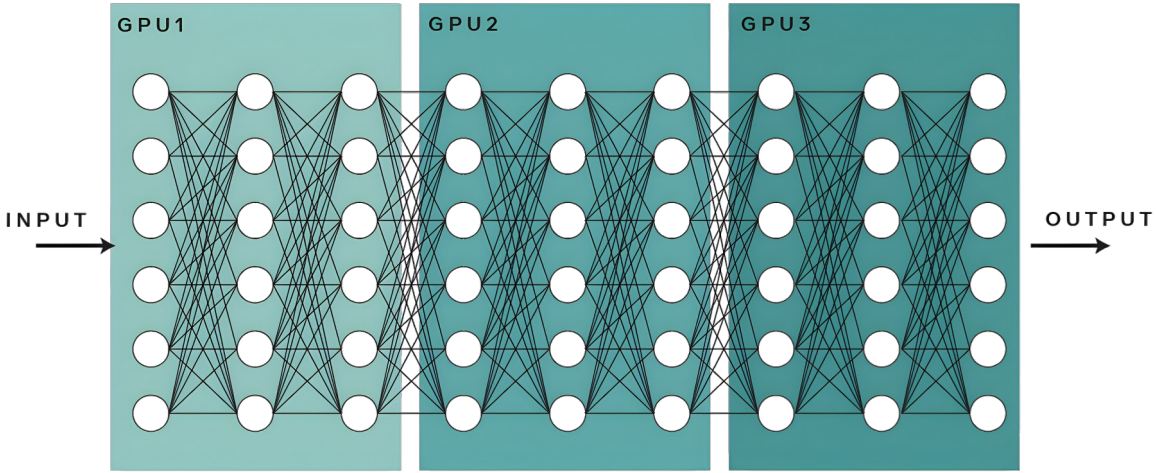
GPU 1

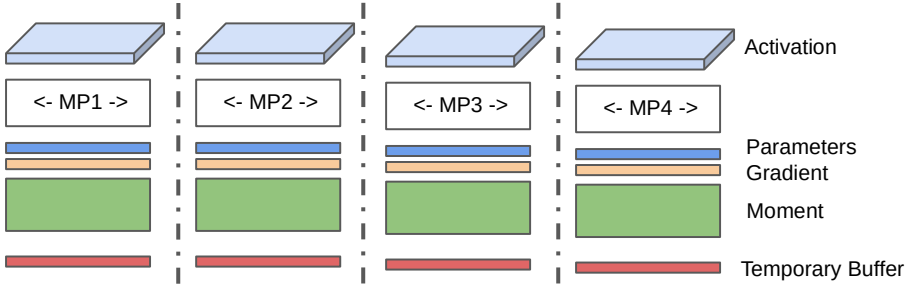
Pipeline Parallelism



[2, 3]

Pipeline Parallelism



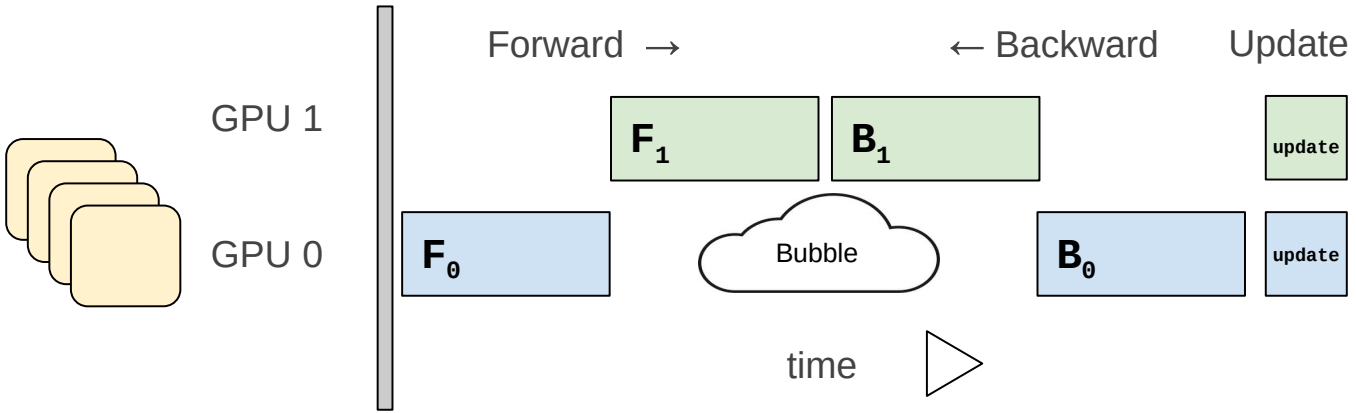


[2, 3]

Advantage

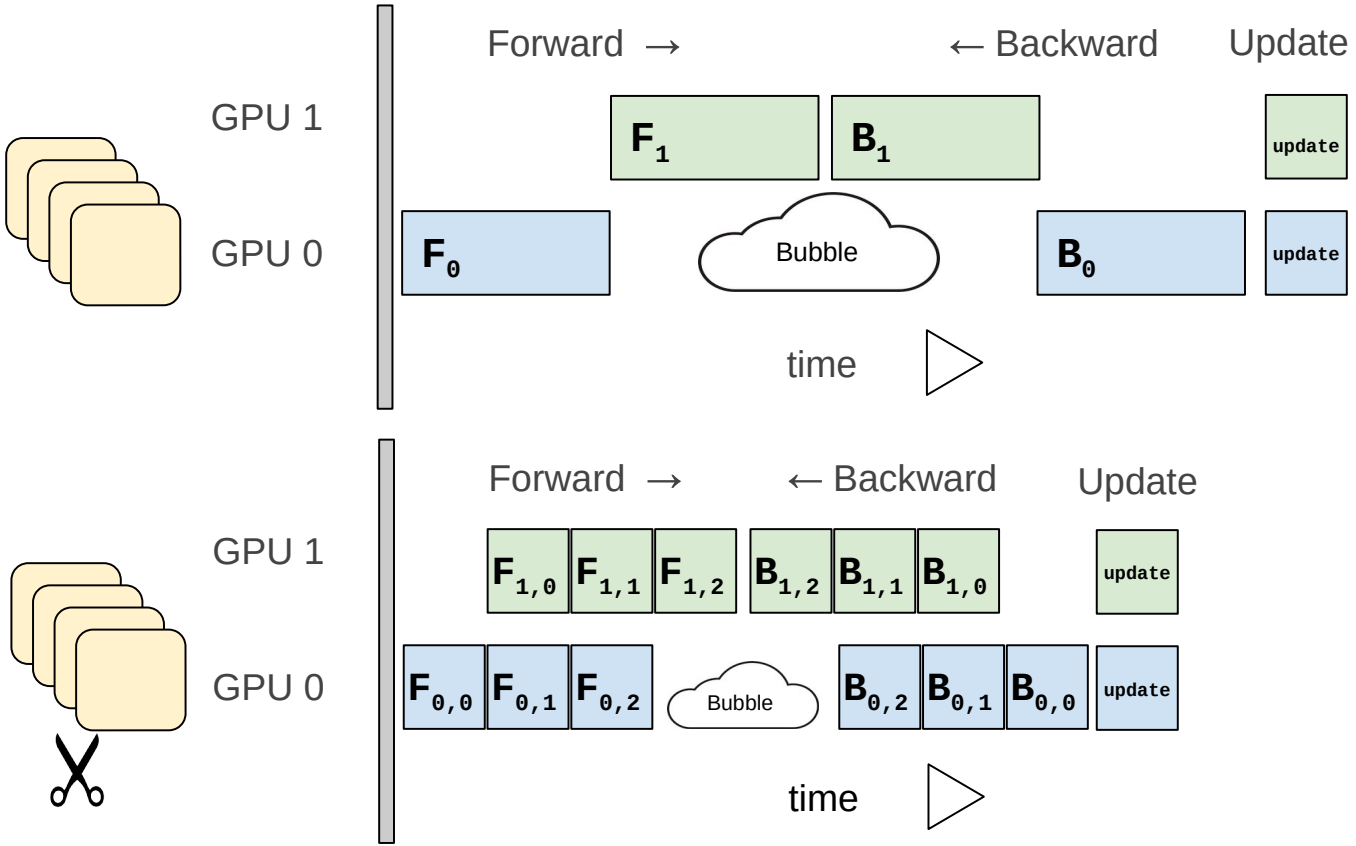
- Distributed memory footprint
- Low communication overhead
- No need to wait before sending the next batch

PP : Micro-batching



Conventional batching

PP : Micro-batching



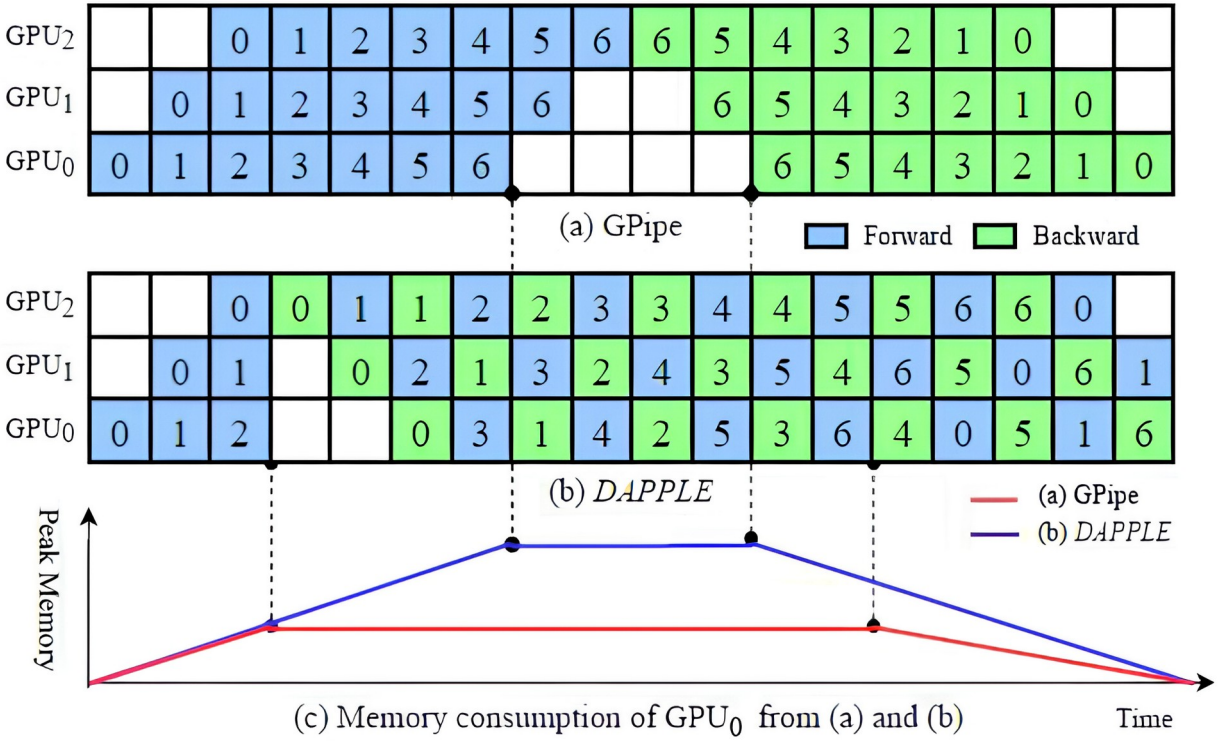
Conventional batching

Micro batching

PP : Optimization with micro-batching

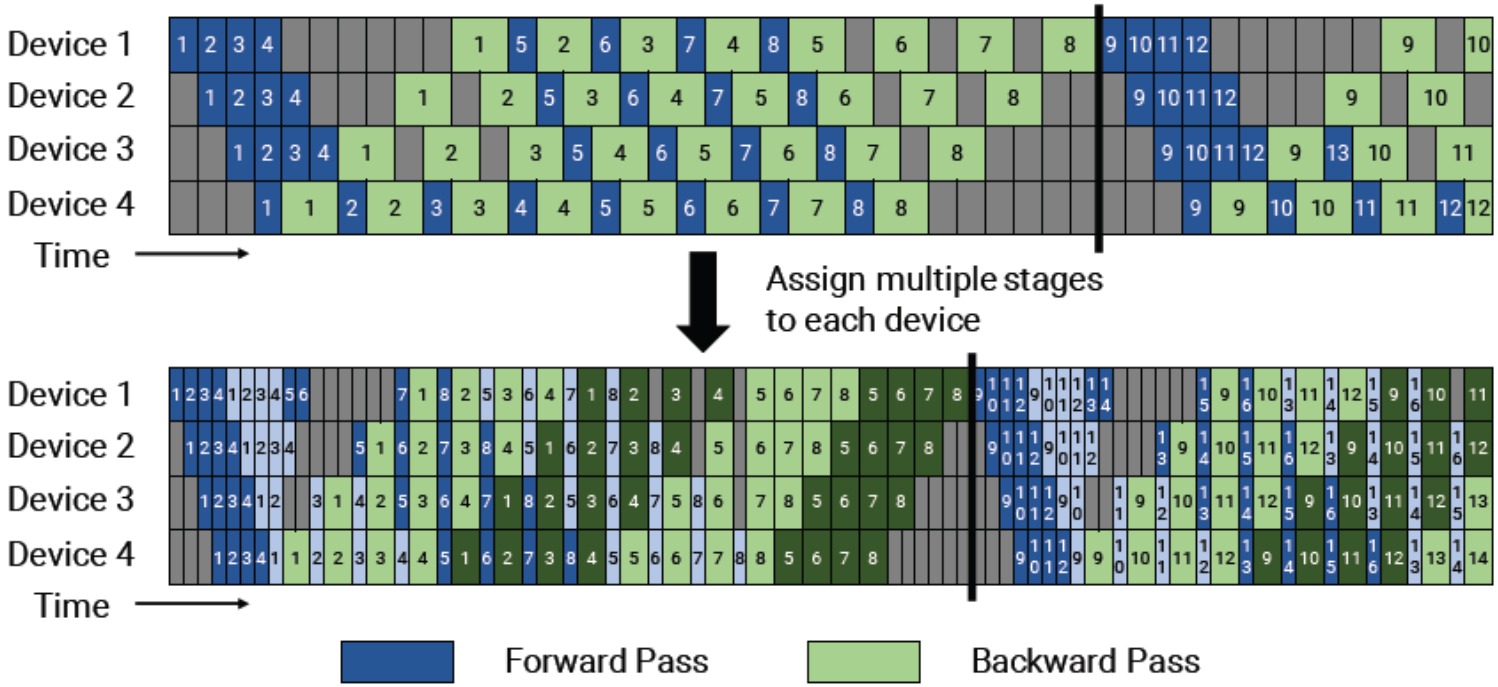
All Forward All Backward

One Forward One Backward



[4, 5]

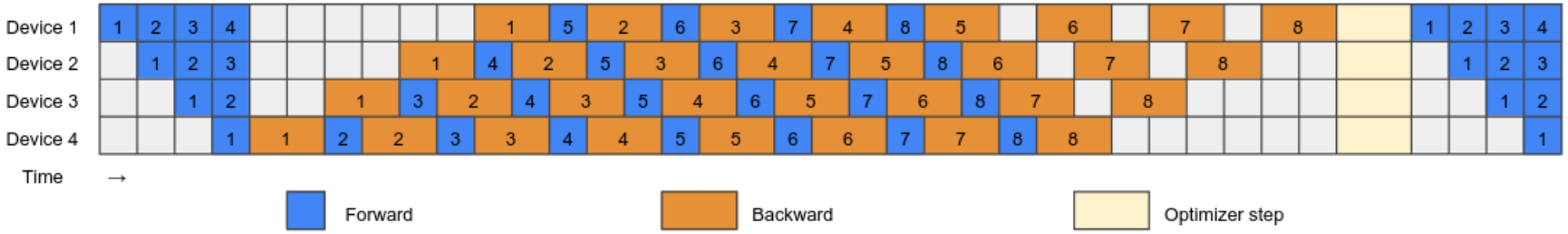
PP : Interleaved 1F1B



Each GPU can execute interleave microbatch of the pipeline.

[4, 5]

PP : Zero Bubble



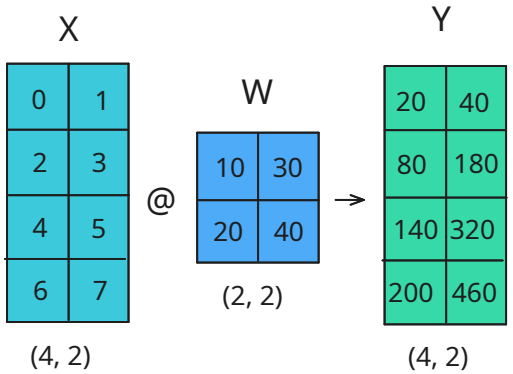
1F1B pipeline schedule.

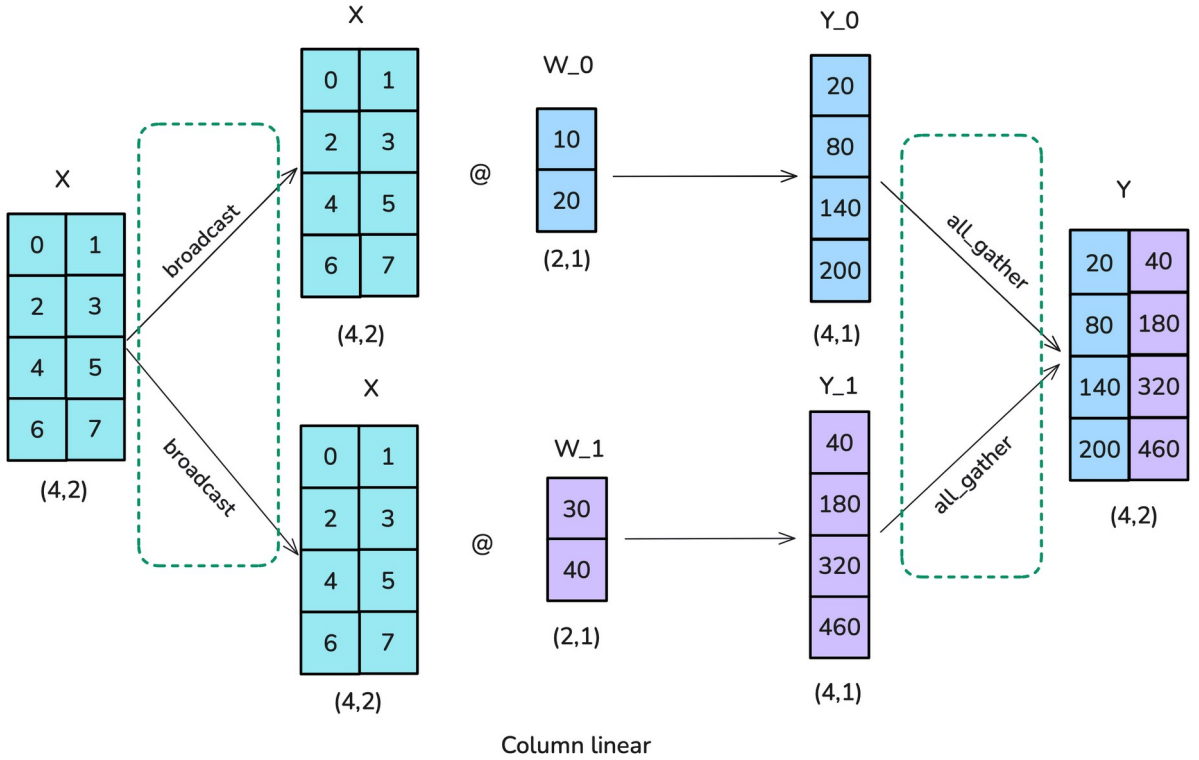


Zero Bubble

[4, 5]

TP : Columns

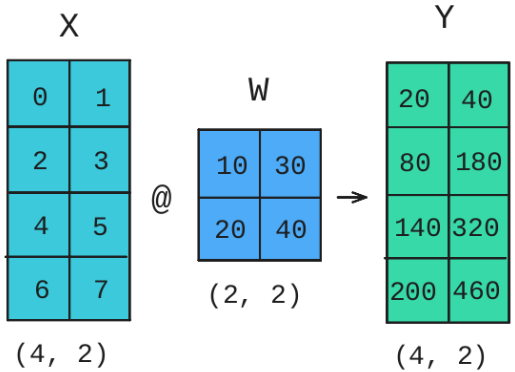


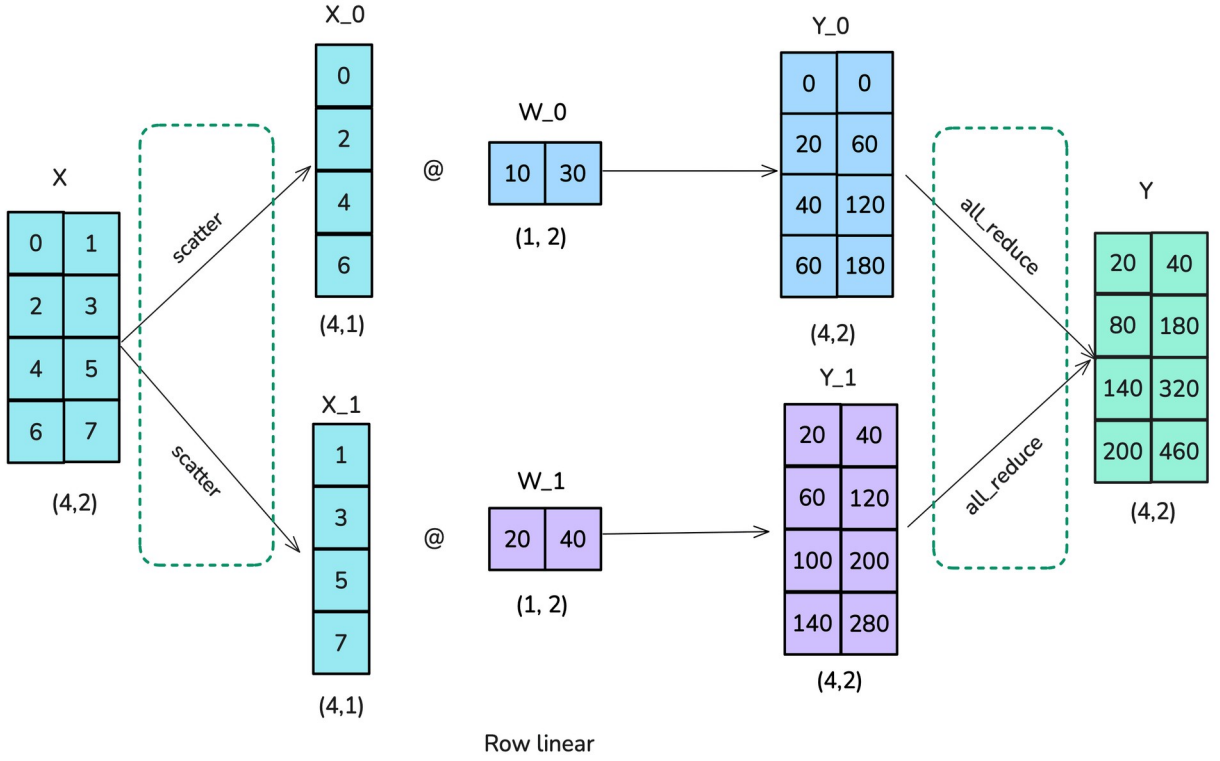


Drawback: Tensor parallelism requires synchronization per layer

[22]

TP : Rows

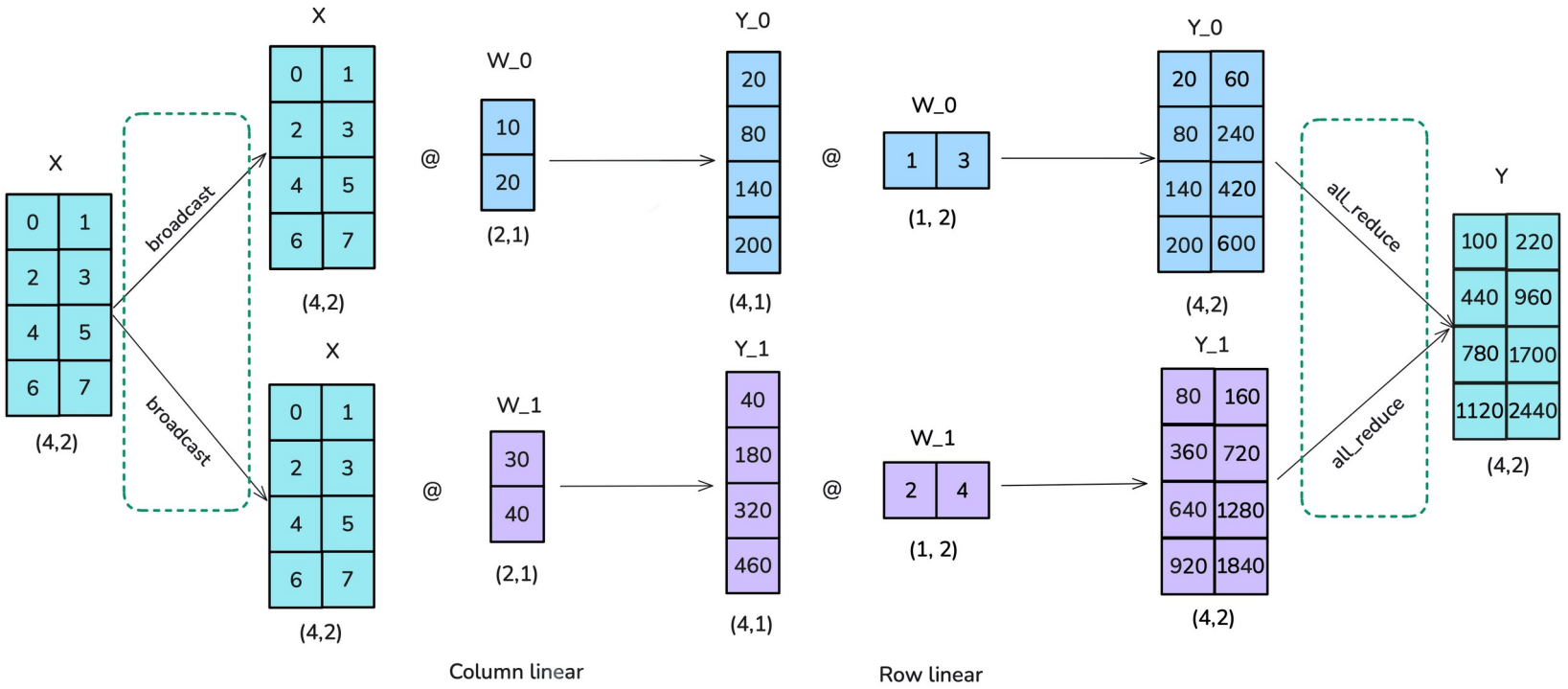




Drawback: Tensor parallelism requires synchronization per layer

[22]

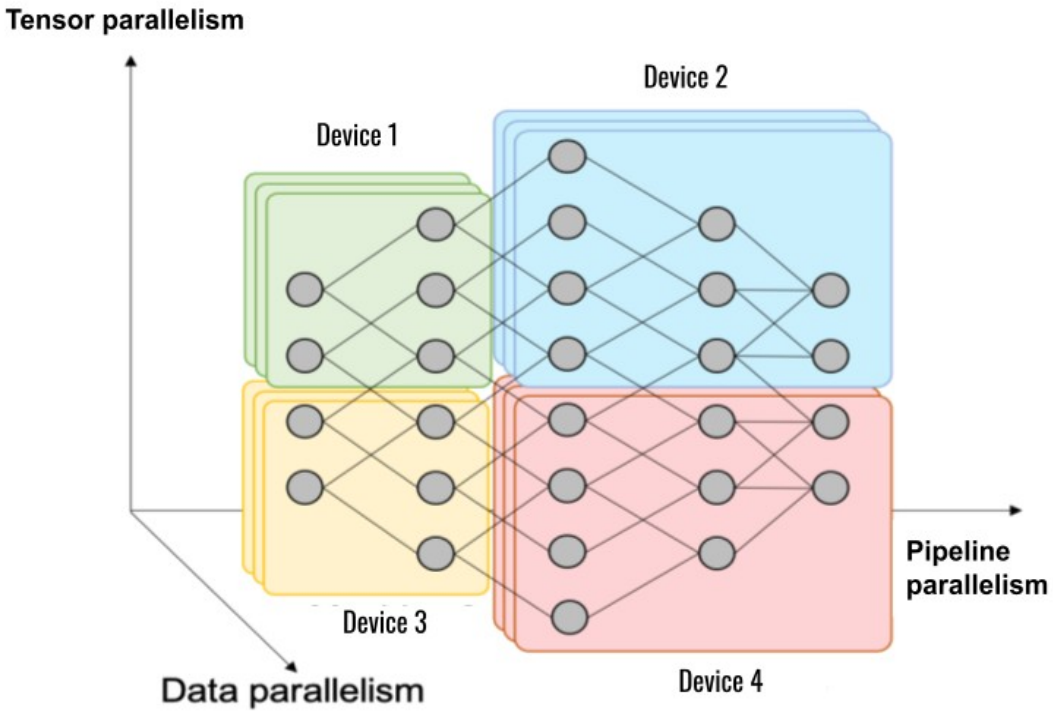
TP : Two Layers, One Synchronization



By alternating column and row parallelism, we only need to synchronize once every two layers

[22]

3D Parallelism



- **Data Parallelism**
  - Easy to implement
  - Better performance
  - Increases batch size
  - (convergence issue)
- **Pipeline Parallelism**
  - Difficult to implement
  - Balance between memory, performance, and convergence
- **Tensor Parallelism**
  - Difficult to implement
  - Good computation speedup
  - **High bandwidth demand**
  - (Intra-node implementation)

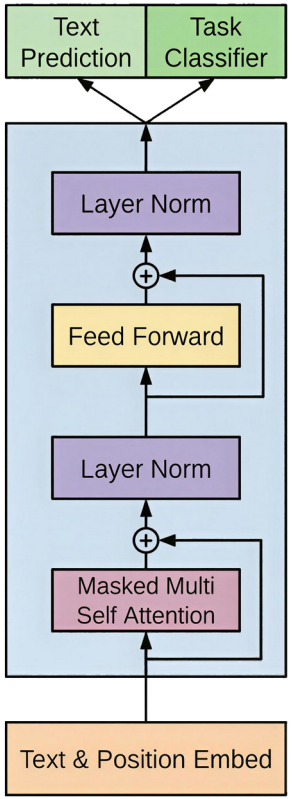
Optimizing the learning of Large Language Model

Context parallelism ◂

Sequence parallelism ◂

Expert parallelism ◂

Transformer Block



Sequence Parallelism

-Utilize Tensor and Pipeline Parallelism

Context Parallelism

-Complexity depends on the number of tokens
-Processing 100 million tokens can requires over 1000GB

CP: Blockwise Parallel Transformer

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$= \frac{\exp\left(\frac{QK^T}{\sqrt{d_k}}\right)V}{\sum \exp\left(\frac{QK^T}{\sqrt{d_k}}\right)}$$

Normalization constant

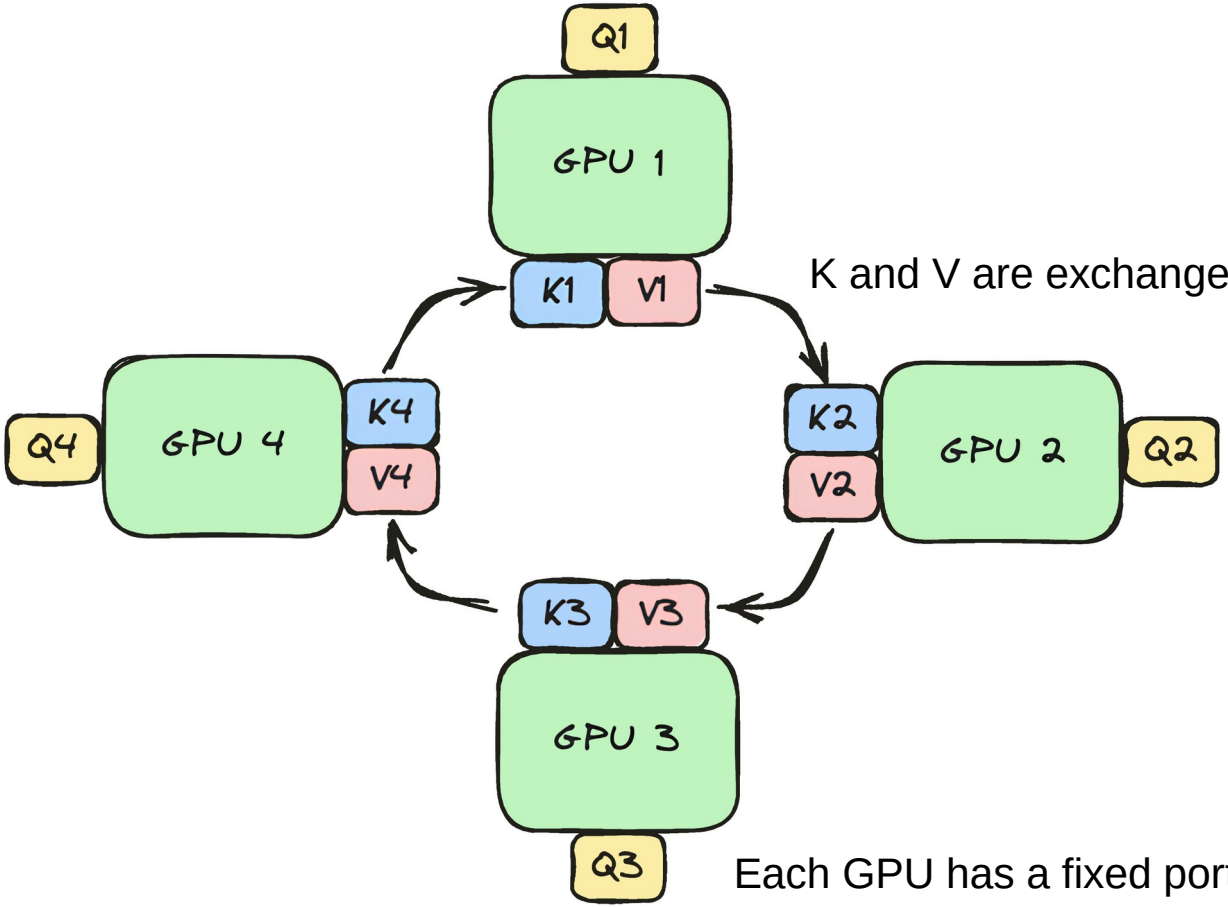
$$\left[\begin{array}{c|c|c} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \end{array}\right] \qquad \left[\begin{array}{c|c} V_{11} & V_{12} \\ V_{21} & V_{22} \\ V_{31} & V_{32} \end{array}\right]$$

$$\left[\begin{array}{cc} Q_{11} & Q_{12} \\ Q_{21} & Q_{22} \\ Q_{31} & Q_{32} \end{array}\right] \left[\begin{array}{ccc} QK^t_{11} & QK^t_{12} & QK^t_{13} \\ QK^t_{21} & QK^t_{22} & QK^t_{23} \\ QK^t_{31} & QK^t_{32} & QK^t_{33} \end{array}\right] \xrightarrow{\text{Exp}} \left[\begin{array}{cc} A_{11} & A_{12} \\ A_{21} & A_{22} \\ A_{31} & A_{32} \end{array}\right]$$

⚠ Attention is not normalized

If I want the first row of the attention matrix, I just need the first row of Q

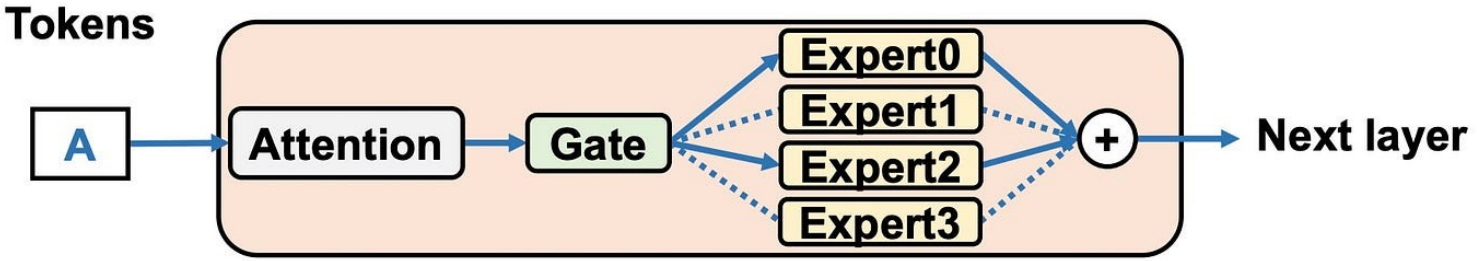
CP: Ring Attention



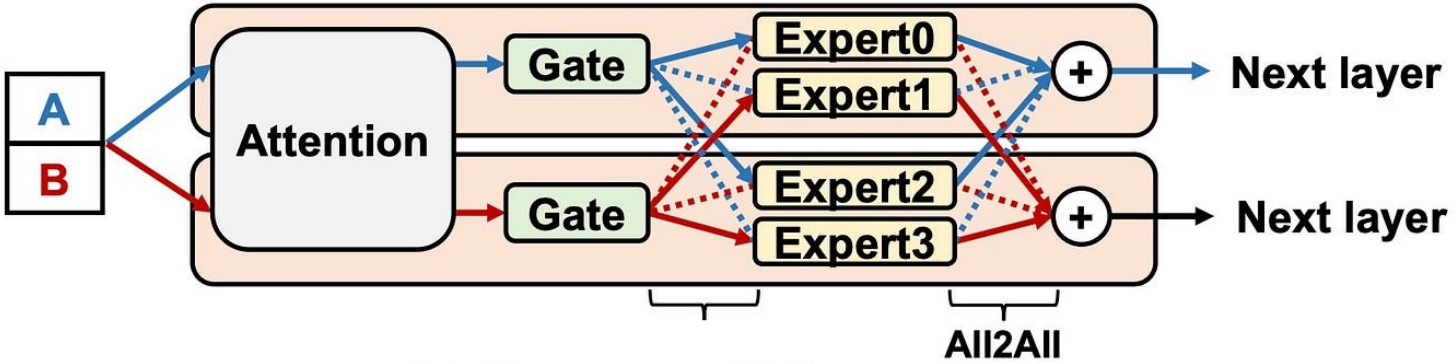
K and V are exchanged between GPUs

Each GPU has a fixed portion of Q

Expert Parallelism

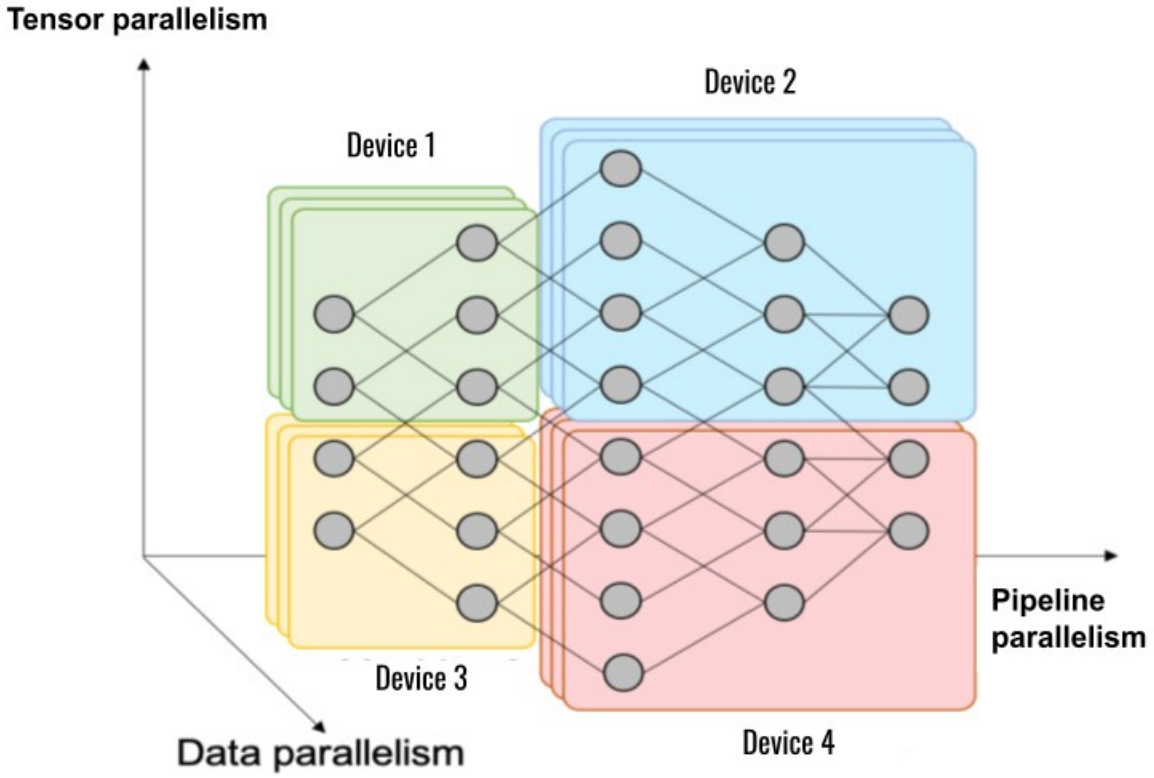


(a) MoE layer.



(b) Expert parallelism.

5D Parallelism



- Data Parallelism
- Pipeline Parallelism
- Tensor Parallelism
- Context Parallelism
- Expert Parallelism

API for Large Models

Deepspeed ◂
Megatron-LM ◂
Pytorch TorchTitan ◂
Ray+vLLM ◂

Deepspeed

Model Scale

Support 200B
Toward 100 Trillion

Speed

Up to 10x faster

Scalability

Superlinear speedup

Usability

Few lines of code changes

```
# Include DeepSpeed configuration arguments
parser = deepspeed.add_config_arguments(parser)
```

```
# Initialize DeepSpeed to use the following features
# 1) Distributed model
# 2) DeepSpeed optimizer
model_engine, optimizer, _, _ = deepspeed.initialize(
                                    args=args, model=model,
                                    model_parameters=parameters,
                                    optimizer=optimizer)
```

```
for step, batch in enumerate(data_loader):
    #forward() method
    loss = model_engine(batch)

    #runs backpropagation
    model_engine.backward(loss)

    #weight update
    model_engine.step()
```

```
{
   "zero_optimization": {
      "stage":2,
      "contiguous_gradients": true,
      "overlap_comm": true,
      "reduce_scatter": true,
      "reduce_bucket_size": 5e8,
      "allgather_bucket_size": 5e8
   }
}
```

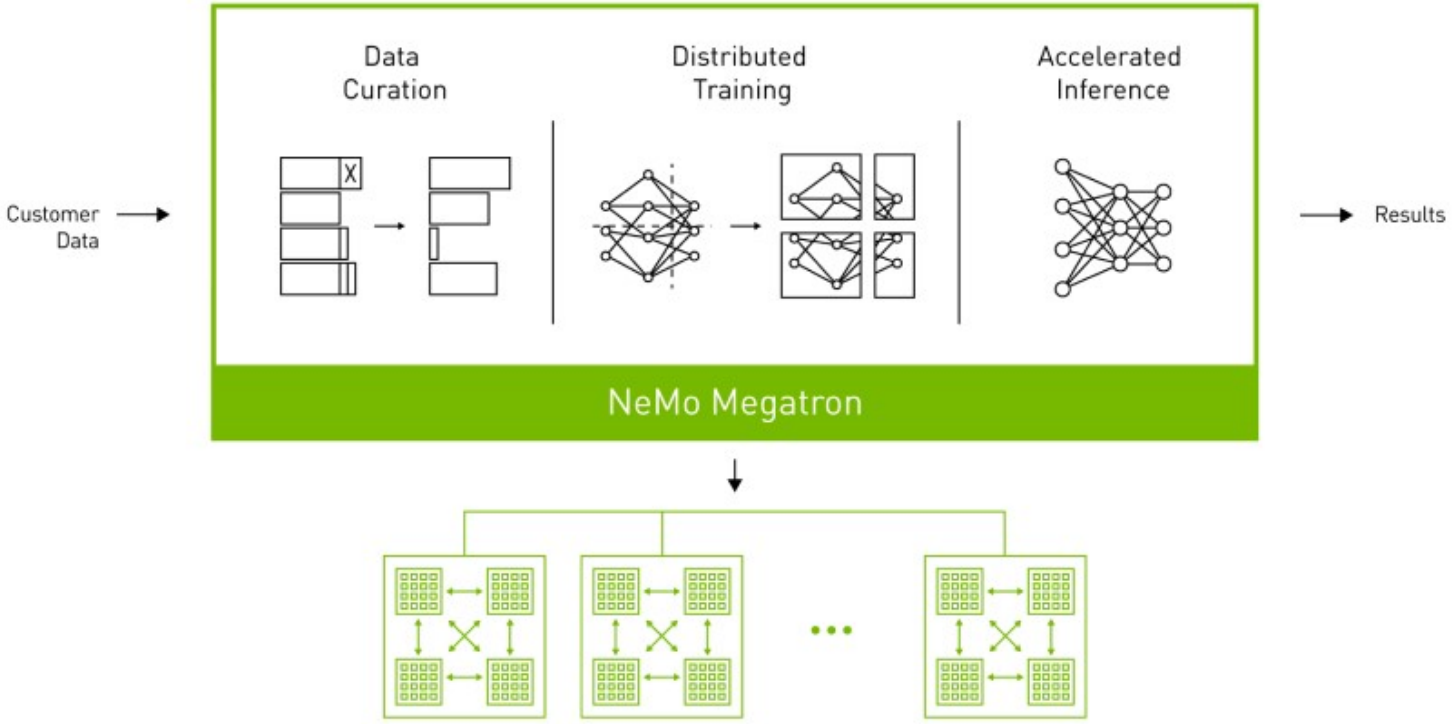
```
# SLURM Job submission
srun train.py -b 28 -s 200 --image-size 288
 —deepspeed --deepspeed_config
ds_config_zero2.json
```

Toutes les applications de Deepspeed

- **Distributed Training with Mixed Precision**
  - 16-bit mixed precision
  - Single-GPU/Multi-GPU/Multi-Node
- **Model Parallelism**
  - Support for Custom Model Parallelism
  - **Integration with Megatron-LM**
- **Pipeline Parallelism**
  - 3D Parallelism
- **The Zero Redundancy Optimizer (ZeRO)**
  - Optimizer State and Gradient Partitioning
  - Activation Partitioning
  - Constant Buffer Optimization
  - Contiguous Memory Optimization
- ZeRO-Offload
  - Leverage both CPU/GPU memory for model training
  - Support 10B model training on a single GPU
- Ultra-fast dense transformer kernels
- Sparse attention
  - Memory- and compute-efficient sparse kernels
  - Support 10x longer sequences than dense
  - Flexible support to different sparse structures
- **1-bit Adam and 1-bit LAMB**
  - Custom communication collective
  - Up to 5x communication volume saving

- Additional Memory and Bandwidth Optimizations
  - Smart Gradient Accumulation
  - Communication/Computation Overlap
- Training Features
  - Simplified training API
  - Gradient Clipping
  - Automatic loss scaling with mixed precision
- Training Optimizers
  - Fused Adam optimizer and arbitrary torch.optim.Optimizer
  - Memory bandwidth optimized FP16 Optimizer
  - Large Batch Training with LAMB Optimizer
  - Memory efficient Training with ZeRO Optimizer
  - CPU-Adam
- Training Agnostic Checkpointing
- Advanced Parameter Search
  - Learning Rate Range Test
  - 1Cycle Learning Rate Schedule
- Simplified Data Loader
- Performance Analysis and Debugging

Nemo Megatron



TorchTitan



Pytorch

torch.distributed.pipelining

torch.distributed.tensor.parallel



VLLM



The best library specifically designed for LLM inference

Références des images utilisées et articles

1. HuggingFace 2021, https://huggingface.co/blog/large-language-models
2. Nicolae, Bogdan, et al. "Deepfreeze: Towards scalable asynchronous checkpointing of deep learning models." *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*. IEEE, 2020.
3. FairScale authors. (2021). FairScale: A general purpose modular PyTorch library for high performance and large scale training. https://fairscale.readthedocs.io/en/latest/deep_dive/pipeline_parallelism.html
4. Fan, Shiqing, et al. "DAPPLE: A pipelined data parallel approach for training large models." *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 2021.
5. Narayanan, Deepak, et al. "PipeDream: Generalized pipeline parallelism for DNN training." *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 2019.
6. Rajbhandari, Samyam, et al. "Zero: Memory optimizations toward training trillion parameter models." *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020.
7. Jiang, Zixuan, et al. "Optimizer Fusion: Efficient Training with Better Locality and Parallelism." *arXiv preprint arXiv:2104.00237* (2021).
8. Deepspeed 2020, https://www.deepspeed.ai/2020/09/08/onebit-adam-blog-post.html
9. Tang, Hanlin, et al. "1-bit adam: Communication efficient large-scale training with adam's convergence speed." *International Conference on Machine Learning*. PMLR, 2021.
10. Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).
11. Peltarion, https://peltarion.com/blog/data-science/self-attention-video
12. Dosovitskiy, Alexey, et al. "An image is worth 16x16 words: Transformers for image recognition at scale." *arXiv preprint arXiv:2010.11929* (2020).
13. Dai, Zihang, et al. "Coatnet: Marrying convolution and attention for all data sizes." *Advances in Neural Information Processing Systems* 34 (2021): 3965-3977.
14. Medium, https://medium.com/@oskyhn_77789/current-convolutional-neural-networks-are-not-translation-equivariant-2f04bb9062e3
15. Huang, Yanping, et al. "Gpipe: Efficient training of giant neural networks using pipeline parallelism." *Advances in neural information processing systems* 32 (2019).
16. Harlap, Aaron, et al. "Pipedream: Fast and efficient pipeline parallel dnn training." *arXiv preprint arXiv:1806.03377* (2018).
17. Fan, Shiqing, et al. "DAPPLE: A pipelined data parallel approach for training large models." *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 2021.
18. Li, Shigang, et al. "Chimera: efficiently training large-scale neural networks with bidirectional pipelines." *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2021.
19. Narayanan, Deepak, et al. "Memory-efficient pipeline-parallel dnn training." *International Conference on Machine Learning*. PMLR, 2021.
20. Narayanan, Deepak, et al. "PipeDream: Generalized pipeline parallelism for DNN training." *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 2019.
21. Li, Shigang, et al. "Chimera: efficiently training large-scale neural networks with bidirectional pipelines." *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2021.
22. Li, Shigang, et al. "Chimera: efficiently training large-scale neural networks with bidirectional pipelines." *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2021.
23. Li, Shigang, et al. "Chimera: efficiently training large-scale neural networks with bidirectional pipelines." *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2021.
24. Li, Shigang, et al. "Chimera: efficiently training large-scale neural networks with bidirectional pipelines." *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2021.
25. Li, Shigang, et al. "Chimera: efficiently training large-scale neural networks with bidirectional pipelines." *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2021.
26. Li, Hao, et al. "Pruning filters for efficient convnets." *arXiv preprint arXiv:1608.08710* (2016).
27. Sanh, Victor, et al. "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter." *arXiv preprint arXiv:1910.01108* (2019).
28. https://lightning.ai/docs/fabric/stable/
29. https://pytorch.org/blog/introducing-pytorch-fully-sharded-data-parallel-api/
30. Hao Liu "Ring Attention with Blockwise Transformers for Near-Infinite Context." *arXiv preprint arXiv:2310.01889* (2023).